PM

Pub Mail

# Deploy do Pub Mail

Tutorial completo de instalação em produção

Atualizado em 14/10/2026

> Guia passo a passo, **detalhado e testado**, para colocar o Pub Mail no ar num servidor Linux limpo — do domínio até o primeiro login. Foi escrito a partir da instalação real feita em **25/09/2026** (Hostinger VPS, Ubuntu 24.04, domínio `reveleonline.com`) e inclui **todos os comandos, arquivos de configuração e armadilhas** encontradas no caminho.
>
> Convenções usadas neste guia:
>
> - `SEU-DOMINIO` = domínio da plataforma (ex.: `reveleonline.com`). A API fica em `api.SEU-DOMINIO`.
> - `IP-DO-SERVIDOR` = IPv4 público do servidor.
> - Comandos com `#` são executados **no servidor, como root**. Comandos com `$` são executados **na sua máquina**.
> - Nunca cole segredos (senhas, tokens, chaves) em chats, prints ou commits. Tudo que é segredo fica **só no servidor**.

---

# 0. Como usar este tutorial (leia antes)

Este guia foi escrito para ser seguido **de cima para baixo, sem pular etapas**, por alguém que nunca fez deploy antes. Tempo total estimado: **1h30 a 2h** (a maior parte é esperar instalações e builds).

## O que você precisa saber para acompanhar

- **Terminal**: no Mac, o app *Terminal* (ou *iTerm*). No Windows, use o **PowerShell** ou o **Windows Terminal** — o comando `ssh` já vem instalado no Windows 10/11. No Linux, qualquer terminal.
- **SSH**: é o "controle remoto" do servidor. Depois de conectar (`ssh root@IP`), tudo que você digita roda **lá**, não na sua máquina.
- **`#` e `$` no começo das linhas** não fazem parte do comando — são só para indicar *onde* rodar (`#` = no servidor como root; `$` = na sua máquina). Não os copie.
- **Blocos que começam com `cat > arquivo <<'EOF2'` e terminam com `EOF2`**: isso cria um arquivo com o conteúdo entre as duas linhas. Copie o bloco **inteiro** (da linha do `cat` até a linha `EOF2`) e cole de uma vez no terminal; ele só executa quando encontra o `EOF2` final.
- **Comandos de várias linhas terminados em `\`**: também cole o bloco inteiro; a barra invertida no fim da linha significa "continua na próxima".
- **Editar um arquivo à mão**: `nano /caminho/do/arquivo` → edite → `Ctrl+O` (salvar) → `Enter` → `Ctrl+X` (sair).
- **Ver um arquivo**: `cat /caminho/do/arquivo`.
- **Segredos**: senhas e tokens só aparecem uma vez, no servidor. Não os copie para chats, prints, e-mails ou commits. Quando este guia mostra `$(grep ^DB_PASS= /root/pubmail.secrets | cut -d= -f2)`, é justamente para **usar** a senha sem **exibi-la**.

## Se algo der errado

1. Leia a mensagem de erro inteira (normalmente a causa está na última linha).
2. Procure o sintoma na seção 24 — Solução de problemas: todos os erros encontrados na instalação real estão lá, com a correção.
3. Cada seção tem um bloco ✅ **Resultado esperado**. Não avance enquanto o resultado não bater.
4. Nada aqui é destrutivo para o servidor: você pode repetir uma seção. A única exceção é a seção 7 (segredos): não gere segredos de novo depois que os `.env` existirem, senão o banco e os serviços deixam de se falar (se precisar, refaça a seção 11 também).

# Sumário


1. Visão geral da arquitetura
2. Pré-requisitos
3. DNS (Cloudflare)

4. Acesso ao servidor e hardening do SSH
5. Pacotes do sistema (Node 22, MySQL, Redis, nginx, certbot, ffmpeg)
6. Swap
7. Segredos da instalação
8. MySQL
9. Redis
10. Código da aplicação em `/opt/pub-mail`
11. Arquivos `.env` (os 4 serviços, variável por variável)
12. Back-end: dependências, migrations, build
13. Seed: usuário SUPER_ADMIN e configurações da plataforma
14. Micro-serviços (IA e e-mail)
15. Front-end (build)
16. Serviços `systemd`
17. nginx
18. Firewall (ufw)
19. HTTPS com certbot (Let's Encrypt)
20. Verificação final (checklist)
21. Pós-instalação (o que configurar dentro do painel)
22. WhatsApp (OpenWA em Docker) — opcional
23. Atualizando o sistema (redeploy de front, back, micros e plugin WordPress)
24. Operação do dia a dia (logs, banco, backup)
25. Solução de problemas (todas as armadilhas conhecidas)
26. Instalando uma segunda cópia (outra empresa / outro domínio)

---

## 1. Visão geral da arquitetura

O Pub Mail é composto por **4 aplicações Node** e **2 serviços de infraestrutura**, todos rodando **no mesmo servidor**, sem Docker:

| Serviço | Pasta no repo | Tecnologia | Porta | Exposto na internet? |
|---|---|---|---|---|
| **back** (API) | `back/` | NestJS 11 + Prisma 7 (MySQL) | 8000 | ✅ via nginx em `https://api.SEU-DOMINIO` |
| **front** (painel + chats públicos) | `front/vite/` | React 19 + Vite 7 (SPA estática) | — | ✅ nginx serve a pasta `dist/` em `https://SEU-DOMINIO` |
| **ai-micro-services** | `ai-micro-services/` | Express (IA dos webchats/e-mails) | 4000 | ❌ só `127.0.0.1` (chamado pelo back) |
| **email-micro-services** | `email-micro-services/` | Express + BullMQ (disparos) | 9999 | ❌ só `127.0.0.1` (chamado pelo back) |
| **MySQL 8** | — | banco `pubmail` | 3306 | ❌ só `127.0.0.1` |
| **Redis 7** | — | filas e cache | 6379 | ❌ só `127.0.0.1` |

Fluxo: o navegador fala **só com o back** ( `VITE_API_URL` ). O back chama os micros por `127.0.0.1` . O nginx faz o TLS, serve o front e o `/uploads` , e faz proxy do `api.` para a porta 8000.

Além disso o back **gerencia o nginx e o certbot sozinho** para os *domínios de cliente* (webchats e quizzes em domínios próprios): por isso o serviço do back roda como **root** e o servidor precisa ter `certbot` + plugin nginx.

Pastas importantes no servidor:

```
/opt/pub-mail/                código (cópia do repositório, sem .git)
/opt/pub-mail/back/.env       configuração do back (segredos, 600)
/opt/pub-mail/back/uploads/   arquivos públicos (imagens de e-mail, mídia do Telegram) — servido pelo nginx
/opt/pub-mail/back/protected/ arquivos de PRODUTOS PAGOS do webchat — NUNCA servido pelo nginx
/opt/pub-mail/front/vite/dist/ build do front (o que o nginx serve)
/root/pubmail.secrets         segredos gerados na instalação (600)
/etc/systemd/system/pubmail-*.service
/etc/nginx/sites-available/{pubmail-front,pubmail-api,000-default-deny,000-default-deny-ssl}
```

---

## 2. Pré-requisitos

### Servidor

- **Ubuntu 24.04 LTS** (validado). Não use 26.04: o MySQL 8.4 dele desliga o `mysql_native_password` por padrão e quebra a conexão do back.
- Mínimo recomendado: **2 vCPU, 8 GB RAM, 40 GB de disco**. O build do front precisa de ~4 GB livres de RAM (por isso configuramos swap).
- Acesso **root** via SSH (senha inicial do provedor; vamos trocar por chave).
- IPv4 público fixo.

**Como conseguir um (exemplo Hostinger, usado na instalação real):** hostinger.com.br → *VPS* → plano **KVM 2** (2 vCPU / 8 GB) ou maior → na configuração escolha **"Ubuntu 24.04 LTS"** (imagem *limpa*, sem painel/Docker pré-instalado) → defina a **senha do root** → conclua. Em poucos minutos o painel (hPanel → VPS) mostra o **IP** e permite redefinir a senha do root. O mesmo vale para DigitalOcean, Vultr, Contabo, Hetzner etc. — o que importa é *Ubuntu 24.04 limpo + acesso root + IPv4*.

**Firewall do provedor:** alguns provedores têm um firewall **fora** do servidor (Hostinger: hPanel → VPS → *Firewall*; Oracle/AWS: security groups). Se existir e estiver ativo, libere **TCP 22, 80 e 443** nele também; senão o site não abre mesmo com tudo certo no servidor. Na Hostinger ele vem desligado por padrão — só confira.

### Domínio

- Um domínio na Cloudflare (ou outro DNS). Vamos usar o apex ( `SEU-DOMINIO` ) para o painel e `api.SEU-DOMINIO` para a API.
- Um e-mail válido para o Let's Encrypt (avisos de expiração) — o mesmo será usado pelo certbot dos domínios de cliente.

### Na sua máquina

- `ssh` , `tar` , `git` . O código é enviado por `tar | ssh` (ou `git clone` no servidor, se o repositório estiver acessível de lá).

### Contas externas (para depois, não bloqueiam a instalação)

- Chaves de IA (Groq, Mistral, Gemini…), conta Resend (e-mail), bots do Telegram, gateway PIX (Mercado Pago/PushinPay). Tudo isso é cadastrado **dentro do painel**, por organização — ver seção 21.

---

## 3. DNS (Cloudflare)

Se o domínio ainda não está na Cloudflare: crie uma conta gratuita em cloudflare.com → *Add a domain* → digite o domínio → plano **Free** → a Cloudflare mostra dois **nameservers** (ex.: `ana.ns.cloudflare.com` e `bob.ns.cloudflare.com` ) → no painel onde você comprou o domínio (Registro.br, GoDaddy, Hostinger…), troque os nameservers pelos dois da Cloudflare → aguarde a Cloudflare marcar o domínio como *Active* (minutos a algumas horas). Depois vá em **DNS** → **Records**.

Crie **3 registros**, todos com a nuvem **cinza (DNS only)** — o certbot precisa alcançar o servidor diretamente para emitir o certificado:

| Tipo | Nome | Conteúdo | Proxy |
|---|---|---|---|
| A | `@` | `IP-DO-SERVIDOR` | DNS only (cinza) |
| A | `api` | `IP-DO-SERVIDOR` | DNS only (cinza) |
| CNAME | `www` | `SEU-DOMINIO` | DNS only (cinza) |

Não crie registro AAAA (IPv6) a menos que o servidor tenha IPv6 configurado.

Confira a propagação **antes** de seguir (os três precisam responder o IP):

```
$ dig +short A SEU-DOMINIO @1.1.1.1
$ dig +short A api.SEU-DOMINIO @1.1.1.1
$ dig +short www.SEU-DOMINIO @1.1.1.1
```

> **Domínios de cliente** (webchats/quizzes em `chat.dominio-do-cliente.com`): o cliente cria um registro **A** apontando para `IP-DO-SERVIDOR` e clica em "Verificar DNS" no painel — o próprio sistema cria o vhost e o certificado. Esses podem ficar com nuvem laranja.
>
> **E-mail (Resend):** o domínio de envio (ex.: `mail.SEU-DOMINIO`) é configurado só no Resend, com os registros SPF/DKIM/DMARC que ele indicar. Não tem relação com o servidor.

---

# 4. Acesso ao servidor e hardening do SSH

## 4.1 Primeiro acesso (senha do provedor)

```
$ ssh root@IP-DO-SERVIDOR
```

## 4.2 Chave SSH (recomendado — depois desligamos a senha)

Na sua máquina:

```
$ ssh-keygen -t ed25519 -f ~/.ssh/pubmail_prod -C "pubmail-prod" -N ""
$ ssh-copy-id -i ~/.ssh/pubmail_prod.pub root@IP-DO-SERVIDOR
```

Crie um alias em `~/.ssh/config`:

```
Host pubmail-prod
    HostName IP-DO-SERVIDOR
    User root
    IdentityFile ~/.ssh/pubmail_prod
    IdentitiesOnly yes
```

Teste: `ssh pubmail-prod 'hostname'`. Daqui em diante, todos os comandos `#` podem ser executados com `ssh pubmail-prod '...'`.

> **Windows:** o `ssh-copy-id` não existe. Faça manualmente: `type $env:USERPROFILE\.ssh\pubmail_prod.pub | ssh root@IP-DO-SERVIDOR "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"` . O arquivo de config fica em `C:\Users\SEU-USUARIO\.ssh\config` (crie sem extensão).

✅ **Resultado esperado:** `ssh pubmail-prod 'hostname'` imprime o nome do servidor **sem pedir senha**.

### 4.3 Desligar login por senha (só depois de confirmar que a chave funciona!)

No Ubuntu 24.04 o `sshd_config` faz `Include /etc/ssh/sshd_config.d/*.conf` **no topo**, e no sshd vale **o primeiro valor lido**. Provedores (Hostinger, cloud-init) costumam deixar um `50-cloud-init.conf` com `PasswordAuthentication yes`. Por isso o nosso drop-in precisa vir **antes** na ordem alfabética: `00-` .

```
# cat > /etc/ssh/sshd_config.d/00-pubmail.conf <<'EOF2'
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitRootLogin prohibit-password
EOF2
# sshd -t && systemctl reload ssh
# sshd -T | grep -E '^passwordauthentication|^permitrootlogin'
```

A última linha precisa mostrar `passwordauthentication no` . Se mostrar `yes` , algum outro drop-in está vencendo — confira com `grep -Hi passwordauth /etc/ssh/sshd_config.d/*` .

---

## 5. Pacotes do sistema

```
# export DEBIAN_FRONTEND=noninteractive
# apt-get update
# apt-get install -y nginx mysql-server redis-server certbot python3-certbot-nginx \
      ffmpeg libvips42 build-essential git curl ufw
```

O que cada um faz: - `nginx` — TLS, front estático, proxy da API, `/uploads` . - `mysql-server` — MySQL 8.0 (banco). - `redis-server` — filas do micro de e-mail, estado das chaves de IA. - `certbot` + `python3-certbot-nginx` — certificados Let's Encrypt (da plataforma **e** dos domínios de cliente, que o back emite sozinho). - `ffmpeg` — conversão de áudio (Telegram) e vídeo (posts). - `libvips42` — base do `sharp` (otimização de imagens de e-mail e Telegram). - `build-essential` — compilar módulos nativos do Node.

### Node.js 22 LTS + Yarn 4 (corepack)

O projeto exige **Node ≥ 22** (o Prisma 7 e o `@prisma/streams-local` recusam o Node 20, que além disso está em fim de vida) e **Yarn 4** (cada `package.json` fixa `"packageManager": "yarn@4.10.3"` ; o corepack lê isso e baixa a versão certa).

```
# curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
# apt-get install -y nodejs
# corepack enable
# node -v        # v22.x
# cd /tmp && echo '{"packageManager":"yarn@4.10.3"}' > package.json && yarn --version && rm package.json   #
4.10.3
```

✅ **Resultado esperado:** `node -v` → `v22.x.x` ; o teste do yarn → `4.10.3` ; `nginx -v` , `mysqld --version` , `redis-server --version` , `certbot --version` , `ffmpeg -version` todos respondem. `systemctl is-active nginx mysql redis-server` → três vezes `active` .

⚠️ Se `yarn --version` mostrar `1.22.x` dentro de uma pasta do projeto, o `package.json` daquela pasta está sem `packageManager` — o corepack cai no Yarn clássico e o install do back **falha**. Todas as 4 pastas do repositório já têm o campo.

---

## 6. Swap

Servidores VPS costumam vir **sem swap**. O build do front (Vite) chega a usar 3–4 GB de RAM; sem swap ele pode ser morto por falta de memória.

```
# fallocate -l 4G /swapfile
# chmod 600 /swapfile
# mkswap /swapfile
# swapon /swapfile
# echo '/swapfile none swap sw 0 0' >> /etc/fstab
# echo 'vm.swappiness=10' > /etc/sysctl.d/99-pubmail.conf && sysctl -w vm.swappiness=10
# swapon --show
```

✅ **Resultado esperado:** `swapon --show` lista `/swapfile` com `SIZE 4G`.

---

## 7. Segredos da instalação

Todos os segredos são **gerados no servidor** e guardados num arquivo que só o root lê. **Nunca** reaproveite segredos de outra instalação nem os copie de um repositório.

```
# umask 077
# cat > /root/pubmail.secrets <<EOF2
DB_PASS=$(openssl rand -hex 24)
JWT_ACCESS_SECRET=$(openssl rand -hex 32)
JWT_REFRESH_SECRET=$(openssl rand -hex 32)
IA_SERVICE_KEY=$(openssl rand -hex 32)
EMAIL_SERVICE_KEY=$(openssl rand -hex 32)
WP_PLUGIN_SIGNING_KEY=$(openssl rand -hex 32)
EOF2
# chmod 600 /root/pubmail.secrets
# umask 022     # ← IMPORTANTE: volte o umask ao normal antes de criar outros arquivos
```

⚠️ **Armadilha real:** se o `umask 077` continuar ativo, os arquivos criados depois (o `.cnf` do MySQL, as pastas de upload) nascem com permissão 600/700 e o MySQL/nginx **ignoram-nos em silêncio**. Sempre `umask 022` depois deste bloco.

Para ler um segredo num comando sem imprimi-lo na tela, use sempre o padrão:

```
# DB_PASS=$(grep ^DB_PASS= /root/pubmail.secrets | cut -d= -f2)
```

---

# 8. MySQL

## 8.1 Banco e usuário

O back conecta por **TCP em 127.0.0.1** com o plugin `mysql_native_password` (exigência do adapter MariaDB do Prisma). O banco **precisa** ser `utf8mb4_unicode_ci` — as migrations criam tabelas com esse collation e as chaves estrangeiras não casam se o default do servidor for outro.

```
# DB_PASS=$(grep ^DB_PASS= /root/pubmail.secrets | cut -d= -f2)
# mysql -e "CREATE DATABASE IF NOT EXISTS pubmail CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
# mysql -e "CREATE USER IF NOT EXISTS 'pubmail'@'127.0.0.1' IDENTIFIED WITH mysql_native_password BY
'$DB_PASS';"
# mysql -e "GRANT ALL PRIVILEGES ON pubmail.* TO 'pubmail'@'127.0.0.1'; FLUSH PRIVILEGES;"
```

## 8.2 Ajustes de desempenho

```
# cat > /etc/mysql/mysql.conf.d/zz-pubmail.cnf <<'EOF2'
[mysqld]
bind-address = 127.0.0.1
innodb_buffer_pool_size = 2G
max_connections = 200
character-set-server = utf8mb4
collation-server = utf8mb4_unicode_ci
default-time-zone = "+00:00"
EOF2
# chmod 644 /etc/mysql/mysql.conf.d/zz-pubmail.cnf
# systemctl restart mysql
# mysql -e "SELECT @@innodb_buffer_pool_size/1024/1024/1024 AS bp_gb, @@max_connections, @@collation_server,
@@time_zone"
```

- `innodb_buffer_pool_size`: ~25% da RAM (2 GB para 8 GB). Se a consulta mostrar `0.125` (128 MB) o arquivo foi ignorado — quase sempre por permissão (veja o `chmod 644`).
- `max_connections = 200`: o back abre ~5 pools de 10 conexões (≈ 51 conexões em repouso).
- Timezone **UTC**: o sistema converte para Brasília na aplicação (agendamentos de e-mail, horários do Telegram). Não mude.

## 8.3 Teste de conexão como a aplicação

```
# mysql -upubmail -p"$DB_PASS" -h127.0.0.1 pubmail -e "SELECT 1"
```

✅ **Resultado esperado:** a consulta da 8.2 mostra `bp_gb = 2.0`, `max_connections = 200`, `utf8mb4_unicode_ci`, `+00:00`; e o `SELECT 1` da 8.3 responde `1` (pode aparecer o aviso "Using a password on the command line" — é normal).

---

# 9. Redis

Só loopback, com limite de memória e **sem eviction** (uma fila do BullMQ não pode perder jobs silenciosamente — é melhor falhar alto).

```
# sed -i 's/^# *maxmemory .*/maxmemory 2gb/; s/^maxmemory .*/maxmemory 2gb/' /etc/redis/redis.conf
# sed -i 's/^# *maxmemory-policy .*/maxmemory-policy noeviction/; s/^maxmemory-policy .*/maxmemory-policy
noeviction/' /etc/redis/redis.conf
# grep -q '^maxmemory 2gb' /etc/redis/redis.conf || echo 'maxmemory 2gb' >> /etc/redis/redis.conf
# grep -q '^maxmemory-policy noeviction' /etc/redis/redis.conf || echo 'maxmemory-policy noeviction' >>
/etc/redis/redis.conf
# grep -q '^bind 127.0.0.1' /etc/redis/redis.conf || sed -i 's/^bind .*/bind 127.0.0.1 -::1/'
/etc/redis/redis.conf
# systemctl restart redis-server
# redis-cli ping                      # PONG
# redis-cli config get maxmemory-policy # noeviction
```

✅ **Resultado esperado:** `PONG` e `noeviction`.

---

## 10. Código da aplicação

### Opção A — enviar da sua máquina (usada na instalação real)

Na raiz do repositório, na sua máquina. O arquivo `.rsync-deploy-excludes` já lista o que **não** deve subir (`node_modules`, `dist`, `.env`, `uploads`, `.git`...):

```
$ ssh pubmail-prod 'mkdir -p /opt/pub-mail'
$ tar -czf - --exclude-from=.rsync-deploy-excludes --exclude='servers.json' --exclude='*.pdf' . \
    | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail'
$ ssh pubmail-prod 'find /opt/pub-mail -name .env -not -path "*/node_modules/*"'   # não deve listar nada
```

### Opção B — clonar no servidor

```
# git clone <URL-DO-REPOSITORIO> /opt/pub-mail
```

Em ambos os casos, confira:

```
# ls /opt/pub-mail
# ls /opt/pub-mail/back/prisma/migrations | wc -l      # dezenas de migrations
# cat /opt/pub-mail/back/.yarnrc.yml                  # nodeLinker: node-modules
```

> `nodeLinker: node-modules` é obrigatório: o padrão do Yarn 4 (Plug'n'Play) quebra o Prisma. Os 4 pacotes já têm o `.yarnrc.yml`.

### Pastas de dados

```
# mkdir -p /opt/pub-mail/back/uploads/email /opt/pub-mail/back/uploads/telegram
# chmod 755 /opt/pub-mail/back/uploads /opt/pub-mail/back/uploads/*
# mkdir -p /opt/pub-mail/back/protected/webchat && chmod 750 /opt/pub-mail/back/protected
```

- `uploads/` é servido **diretamente pelo nginx** (precisa de 755).
- `protected/` guarda os produtos vendidos no webchat; é lido só pelo back e entregue por URL assinada. **Não** pode aparecer em nenhum `location` do nginx.

- `wordpress/tradutor-invisivel/` (vem junto com o código) é o plugin WordPress do módulo **WordPress**: o back monta dele o `.zip` que cada cliente baixa, com a chave da conta embutida. Confira que os scripts compilados existem: `ls /opt/pub-mail/wordpress/tradutor-invisivel/assets/*.min.js` (3 arquivos). Ao atualizar o plugin, envie essa pasta de novo — os sites dos clientes recebem a versão nova pela atualização automática do WordPress. O mesmo vale para `wordpress/push-in-play/` (plugin Push In Play, sem passo de compilação: é PHP + JS puro).

✅ **Resultado esperado:** `ls /opt/pub-mail` mostra `back ai-micro-services email-micro-services front README.md …`; a busca por `.env` não lista nada (os `.env` são criados na seção 11).

---

## 11. Arquivos `.env`

Cada serviço tem o seu `.env`, criado a partir do `.env.example` correspondente. Os quatro ficam com permissão **600** e **nunca** são versionados (o `.gitignore` já os ignora).

O bloco abaixo gera os 4 arquivos de uma vez, lendo os segredos do `/root/pubmail.secrets`. Troque `SEU-DOMINIO` nas duas primeiras linhas.

```
# DOMAIN=SEU-DOMINIO
# API=https://api.$DOMAIN
# umask 077; . /root/pubmail.secrets

# cat > /opt/pub-mail/back/.env <<EOF2
NODE_ENV=production
PORT=8000
DATABASE_HOST=127.0.0.1
DATABASE_PORT=3306
DATABASE_USER=pubmail
DATABASE_PASS=$DB_PASS
DATABASE_NAME=pubmail
DATABASE_URL=mysql://pubmail:$DB_PASS@127.0.0.1:3306/pubmail
DATABASE_POOL_LIMIT=10
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
JWT_ACCESS_SECRET=$JWT_ACCESS_SECRET
JWT_REFRESH_SECRET=$JWT_REFRESH_SECRET
JWT_ACCESS_EXPIRES_IN=24h
JWT_REFRESH_EXPIRES_IN=30d
IA_SERVICE_URL=http://127.0.0.1:4000
IA_SERVICE_KEY=$IA_SERVICE_KEY
EMAIL_SERVICE_URL=http://127.0.0.1:9999
EMAIL_SERVICE_KEY=$EMAIL_SERVICE_KEY
PUBLIC_API_URL=$API
BASE_API_URL=$API
WEBCHAT_FRONT_DIST=/opt/pub-mail/front/vite/dist
WEBCHAT_BACKEND_INTERNAL_URL=http://127.0.0.1:8000
WEBCHAT_DOMAIN_BIND_PROVIDER=CERTBOT
NGINX_SITES_AVAILABLE=/etc/nginx/sites-available
NGINX_SITES_ENABLED=/etc/nginx/sites-enabled
CERTBOT_BIN=certbot
CERTBOT_MODE=nginx
EMAIL_TOKENS_FOR_ONE_LEAD=0
EMAIL_TOKENS_FOR_ONE_TEMPLATE=0
WEBCHAT_TOKENS_PER_MESSAGE=0
SOCIAL_POST_SCHEDULE_TOKENS_COST=0
AVATAR_GENERATION_TOKENS_COST=0
HTTP_BODY_LIMIT=200mb
STRIPE_SECRET_KEY=sk_test_disabled_placeholder
WP_PLUGIN_SIGNING_KEY=$WP_PLUGIN_SIGNING_KEY
EOF2

# cat > /opt/pub-mail/ai-micro-services/.env <<EOF2
PORT=4000
IA_SERVICE_KEY=$IA_SERVICE_KEY
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
OPENROUTER_REFERER=https://$DOMAIN
EOF2

# cat > /opt/pub-mail/email-micro-services/.env <<EOF2
PORT=9999
EMAIL_SERVICE_KEY=$EMAIL_SERVICE_KEY
```

```
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
BASE_URL_API=$API
NEST_WEBHOOK_URL=http://127.0.0.1:8000/webhooks/email
TRACKING_SIGN_SECRET=$JWT_ACCESS_SECRET
EOF2

# cat > /opt/pub-mail/front/vite/.env <<EOF2
VITE_APP_BASE_NAME=/
VITE_API_URL=$API
VITE_APP_VERSION=1.0.0
GENERATE_SOURCEMAP=false
VITE_STRIPE_ENABLED=false
VITE_STRIPE_PUBLISHABLE_KEY=
EOF2

# umask 022
# chmod 600 /opt/pub-mail/back/.env /opt/pub-mail/ai-micro-services/.env /opt/pub-mail/email-micro-
services/.env /opt/pub-mail/front/vite/.env
```

✅ **Resultado esperado:** `ls -l /opt/pub-mail/back/.env /opt/pub-mail/*/.env /opt/pub-mail/front/vite/.env` mostra os 4 arquivos com `-rw-------`; `grep -c . /opt/pub-mail/back/.env` → `35`.

### 11.1 Back (`back/.env`) — variável por variável

| Variável | Valor | Por quê |
|---|---|---|
| `NODE_ENV` | `production` | Desliga logs/validações de desenvolvimento. |
| `PORT` | `8000` | Porta interna da API (o nginx faz proxy para ela). |
| `DATABASE_HOST/PORT/USER/PASS/NAME` | `127.0.0.1` / `3306` / `pubmail` / senha / `pubmail` | **É isso que o runtime usa** (`prisma.service.ts` monta o adapter com estas variáveis). |
| `DATABASE_URL` | `mysql://pubmail:SENHA@127.0.0.1:3306/pubmail` | Usada **só pelo CLI do Prisma** (`migrate deploy`, `generate`). As duas formas precisam coexistir. |
| `DATABASE_POOL_LIMIT` | `10` | **Obrigatório.** O adapter mantém `minimumIdle = connectionLimit` e o back abre ~5 pools; com o padrão (50) ele pede 250 conexões e derruba o MySQL com "Too many connections". |
| `REDIS_HOST/PORT` | `127.0.0.1` / `6379` | Redis local. |
| `JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET` | aleatórios (hex 32) | Assinam os tokens de login. Trocar invalida todas as sessões. |
| `JWT_ACCESS_EXPIRES_IN` / `JWT_REFRESH_EXPIRES_IN` | `24h` / `30d` | Validade dos tokens. |
| `IA_SERVICE_URL` / `IA_SERVICE_KEY` | `http://127.0.0.1:4000` / segredo | Endereço e chave do micro de IA (a mesma chave vai no `.env` do micro). |

| Variável | Valor | Por quê |
|---|---|---|
| `EMAIL_SERVICE_URL` / `EMAIL_SERVICE_KEY` | `http://127.0.0.1:9999` / segredo | Idem para o micro de e-mail. |
| `PUBLIC_API_URL` | `https://api.SEU-DOMINIO` | URL pública da API: webhooks do Telegram e dos gateways PIX, links assinados dos produtos, formulário de captação. |
| `BASE_API_URL` | `https://api.SEU-DOMINIO` | Mesma URL, usada por outros módulos (tracking, uploads). |
| `WEBCHAT_FRONT_DIST` | `/opt/pub-mail/front/vite/dist` | Pasta que os vhosts de domínio de cliente vão servir. |
| `WEBCHAT_BACKEND_INTERNAL_URL` | `http://127.0.0.1:8000` | Usado nos vhosts de cliente para proxy do `/ads.txt`. O padrão do código é `:3000` (errado) — **defina sempre**. |
| `WEBCHAT_DOMAIN_BIND_PROVIDER` | `CERTBOT` | Como provisionar domínio de cliente. Só aceita `CERTBOT`, `NONE`, `DISABLED` ou `MOCK` — qualquer outro valor lança exceção ao clicar "Verificar DNS". |
| `NGINX_SITES_AVAILABLE` / `NGINX_SITES_ENABLED` | caminhos padrão do Debian | Onde o back escreve os vhosts dos clientes. |
| `CERTBOT_BIN` / `CERTBOT_MODE` | `certbot` / `nginx` | Como o back emite certificados dos clientes. |
| `EMAIL_TOKENS_FOR_ONE_LEAD` … `AVATAR_GENERATION_TOKENS_COST` | `0` | Custos internos em "tokens" (carteira). Com `0` nada é cobrado. |
| `HTTP_BODY_LIMIT` | `200mb` | Limite do body JSON/urlencoded do Express. Casa com o `client_max_body_size` do nginx. |
| `STRIPE_SECRET_KEY` | `sk_test_disabled_placeholder` | **Não pode ficar vazio**: o `BillingService` lança erro no boot e o back não sobe. O placeholder é inerte porque o Stripe está desligado no front. |
| `WP_PLUGIN_SIGNING_KEY` | aleatório (hex 32) | Módulo **WordPress**: assina a verificação de licença dos plugins ("Tradutor Invisível" e "Push In Play") instalados nos sites dos clientes. **Nunca troque depois de ter clientes**: os plugins instalados deixariam de aceitar as respostas até atualizarem. Sem ela, o back deriva uma do `JWT_ACCESS_SECRET` (e aí trocar o JWT pararia os plugins). |

| Variável | Valor | Por quê |
|---|---|---|
| `ORG_WEBHOOK_ALLOW_LOCAL_PORTS` (opcional) | `18999` | Webhooks de saída da org só vão para `https://` porta 443; esta lista libera `http://127.0.0.1:<porta>` **só para testes automatizados** (deixe vazio se não roda o E2E). |
| `APP_PUBLIC_URL` (opcional) | `https://SEU-DOMINIO` | Endereço do painel: botão "Abrir o Pub Mail" do plugin WordPress e base do **link de pagamento** das cobranças do WhatsApp ( `/pay/<token>` ). Padrão: `PUBLIC_API_URL` sem o `api.` . |
| `WP_PLUGIN_SOURCE_DIR` (opcional) | `/opt/pub-mail/wordpress/tradutor-invisivel` | Pasta com o código do plugin Tradutor Invisível, de onde o back monta o `.zip` de cada cliente. Padrão: `../wordpress/tradutor-invisivel` a partir de `back/` . |
| `WP_PIP_SOURCE_DIR` (opcional) | `/opt/pub-mail/wordpress/push-in-play` | Idem para o plugin Push In Play (notificações push). Padrão: `../wordpress/push-in-play` a partir de `back/` . A chave da conta é a mesma dos dois plugins. |

### 11.2 Micro de IA ( `ai-micro-services/.env` )

| Variável | Valor |
|---|---|
| `PORT` | `4000` |
| `IA_SERVICE_KEY` | **mesmo valor** do back |
| `REDIS_HOST/PORT` | `127.0.0.1` / `6379` |
| `OPENROUTER_REFERER` | `https://SEU-DOMINIO` (identificação enviada ao OpenRouter) |

### 11.3 Micro de e-mail ( `email-micro-services/.env` )

| Variável | Valor | Por quê |
|---|---|---|
| `PORT` | `9999` | |
| `EMAIL_SERVICE_KEY` | **mesmo valor** do back | |
| `REDIS_HOST/PORT` | `127.0.0.1` / `6379` | Fila BullMQ dos disparos. |
| `BASE_URL_API` | `https://api.SEU-DOMINIO` | Monta o pixel de abertura e os links rastreados dentro dos e-mails (público). |
| `NEST_WEBHOOK_URL` | `http://127.0.0.1:8000/webhooks/email` | Callback interno com o resultado do disparo. O micro concatena `/schedule-sent/:id` — **precisa terminar em** `/webhooks/email` . |
| `TRACKING_SIGN_SECRET` | **mesmo valor** de `JWT_ACCESS_SECRET` do back | Assina os links de pixel de abertura e clique rastreado (o back valida com o mesmo segredo). Sem ela o micro envia links sem assinatura, que o back recusa para e-mails novos. |

### 11.4 Front ( `front/vite/.env` )

Os valores entram **no build** — mudar exige rebuild do front.

| Variável | Valor |
|---|---|
| `VITE_APP_BASE_NAME` | `/` |
| `VITE_API_URL` | `https://api.SEU-DOMINIO` |
| `VITE_APP_VERSION` | `1.0.0` |
| `GENERATE_SOURCEMAP` | `false` |
| `VITE_STRIPE_ENABLED` | `false` |
| `VITE_STRIPE_PUBLISHABLE_KEY` | vazio |

---

## 12. Back-end: dependências, migrations, build

```
# unset CI
# cd /opt/pub-mail/back
# yarn --version                      # 4.10.3 (se mostrar 1.22 → veja a seção 25)
# yarn install
# test -d node_modules/@nestjs/core && echo INSTALL_OK
# yarn prisma migrate deploy          # cria todas as tabelas no banco vazio
# yarn prisma generate                # gera o client em back/generated/prisma
# yarn nest build > /tmp/nest-build.log 2>&1; grep -cE "error TS|Found [0-9]+ error" /tmp/nest-build.log
# test -f dist/src/main.js && echo BUILD_OK
```

Detalhes importantes:

- **`unset CI`** : se a variável `CI` existir no ambiente (mesmo `CI=0` ), o Yarn 4 entra em modo imutável e recusa instalar quando precisa mexer no lockfile.
- O `grep -c` da build **precisa mostrar `0`** . O `nest build` **gera o `dist/` mesmo com erro de TypeScript** — o `test -f dist/src/main.js` sozinho não prova nada.
- `migrate deploy` aplica a cadeia inteira de migrations (todas foram validadas num banco vazio). Confira: `mysql -upubmail -p"$DB_PASS" -h127.0.0.1 pubmail -e "SELECT COUNT(*) FROM _prisma_migrations WHERE finished_at IS NOT NULL"` .

✅ **Resultado esperado:** `INSTALL_OK` ; `migrate deploy` termina com "All migrations have been successfully applied" (dezenas de migrations); `generate` mostra "Generated Prisma Client"; o `grep -c` mostra **`0`** ; `BUILD_OK` . Leva de 3 a 6 minutos no total.

---

## 13. Seed

O seed cria o mínimo para o sistema funcionar: a linha `system_settings` (IP de borda para domínios de cliente + e-mail do certbot), a **organização oculta do desenvolvedor** e o usuário **SUPER_ADMIN**. É **idempotente** (rodar de novo só atualiza a senha) e não guarda senha em arquivo nenhum.

```
# cd /opt/pub-mail/back
# DB_PASS=$(grep ^DB_PASS= /root/pubmail.secrets | cut -d= -f2)
# SEED_ADMIN_EMAIL=admin@SEU-DOMINIO \
  SEED_ADMIN_PASSWORD='UMA-SENHA-FORTE' \
  SEED_ADMIN_NAME='Super Admin' \
  SEED_ORG_NAME='SuperAdmin' \
  SEED_EDGE_IP=IP-DO-SERVIDOR \
  SEED_CERTBOT_EMAIL=voce@SEU-DOMINIO \
  node scripts/seed-platform.mjs | mysql -upubmail -p"$DB_PASS" -h127.0.0.1 pubmail
# mysql -upubmail -p"$DB_PASS" -h127.0.0.1 pubmail -e "SELECT email, role FROM users; SELECT webchat_edge_ip,
certbot_email FROM system_settings;"
```

| Variável | Significado |
|---|---|
| `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` | Login do SUPER_ADMIN (senha com no mínimo 8 caracteres; gravada com bcrypt). |
| `SEED_ADMIN_NAME` | Nome exibido. |
| `SEED_ORG_NAME` | Nome da organização oculta (invisível para as outras organizações na área de administração). |
| `SEED_EDGE_IP` | IP que os clientes apontam nos domínios de webchat/quiz (mostrado na tela de Domínios). |
| `SEED_CERTBOT_EMAIL` | E-mail usado pelo certbot ao emitir certificados dos domínios de cliente (editável depois em Configurações → Integrações). |

✅ **Resultado esperado:** a saída do seed termina com `seed-platform OK`; o `SELECT` mostra seu e-mail com `role = SUPER_ADMIN` e a linha de `system_settings` com o IP e o e-mail.

> O SUPER_ADMIN cria as demais organizações e usuários em **Administração → Usuários**.

---

## 14. Micro-serviços

```
# unset CI
# cd /opt/pub-mail/ai-micro-services && yarn install && test -d node_modules/express && echo AI_OK
# cd /opt/pub-mail/email-micro-services && yarn install && test -d node_modules/bullmq && echo EMAIL_OK
```

Não há build: rodam direto (`node src/server.js` e `node server.js`).

✅ **Resultado esperado:** `AI_OK` e `EMAIL_OK`.

---

## 15. Front-end

```
# unset CI
# cd /opt/pub-mail/front/vite
# yarn install
# export NODE_OPTIONS=--max-old-space-size=4096
# yarn build                                  # gera dist/
# test -f dist/index.html && echo FRONT_OK
# grep -l "api.SEU-DOMINIO" dist/assets/*.js | head -1     # confirma que a URL da API entrou no build
```

O build leva ~20–60 s e usa bastante memória (daí o `NODE_OPTIONS` e o swap).

✅ **Resultado esperado:** `✓ built in NNs`, `FRONT_OK`, e o `grep` devolve o nome de um arquivo `.js` (prova de que o front vai chamar a API certa).

---

## 16. Serviços `systemd`

Três units, todas com `Restart=always` e lendo o `.env` de cada serviço. O back roda como **root** de propósito: ele escreve vhosts em `/etc/nginx/sites-*` e executa o `certbot` para os domínios de cliente.

```
# cat > /etc/systemd/system/pubmail-back.service <<'EOF2'
[Unit]
Description=Pub Mail - API (NestJS :8000)
After=network.target mysql.service redis-server.service
Wants=mysql.service redis-server.service

[Service]
Type=simple
User=root
WorkingDirectory=/opt/pub-mail/back
EnvironmentFile=/opt/pub-mail/back/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node dist/src/main.js
Restart=always
RestartSec=3
LimitNOFILE=65536
StandardOutput=journal
StandardError=journal
SyslogIdentifier=pubmail-back

[Install]
WantedBy=multi-user.target
EOF2

# cat > /etc/systemd/system/pubmail-ai.service <<'EOF2'
[Unit]
Description=Pub Mail - AI micro-service (:4000)
After=network.target mysql.service redis-server.service
Wants=mysql.service redis-server.service

[Service]
Type=simple
User=root
WorkingDirectory=/opt/pub-mail/ai-micro-services
EnvironmentFile=/opt/pub-mail/ai-micro-services/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node src/server.js
Restart=always
RestartSec=3
LimitNOFILE=65536
StandardOutput=journal
StandardError=journal
SyslogIdentifier=pubmail-ai

[Install]
WantedBy=multi-user.target
EOF2

# cat > /etc/systemd/system/pubmail-email.service <<'EOF2'
[Unit]
Description=Pub Mail - Email micro-service (:9999)
After=network.target mysql.service redis-server.service
Wants=mysql.service redis-server.service
```

```
[Service]
Type=simple
User=root
WorkingDirectory=/opt/pub-mail/email-micro-services
EnvironmentFile=/opt/pub-mail/email-micro-services/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=3
LimitNOFILE=65536
StandardOutput=journal
StandardError=journal
SyslogIdentifier=pubmail-email

[Install]
WantedBy=multi-user.target
EOF2

# systemctl daemon-reload
# systemctl enable --now pubmail-back pubmail-ai pubmail-email
# sleep 8
# systemctl is-active pubmail-back pubmail-ai pubmail-email       # active active active
# ss -ltn | grep -E ':8000|:4000|:9999'
# journalctl -u pubmail-back -n 30 --no-pager | grep -E "Nest application successfully|ERROR"
```

Teste rápido da API por dentro do servidor (400 = está respondendo e validando):

```
# curl -s -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/auth/login -H 'Content-Type:
application/json' -d '{}'
```

✅ **Resultado esperado:** `active active active`; as três portas aparecem no `ss`; o `journalctl` mostra `Nest application successfully started`; o `curl` responde `400`. Se um serviço estiver `failed`, veja o motivo com `journalctl -u pubmail-back -n 50 --no-pager` (os erros mais comuns estão na seção 25).

---

## 17. nginx

Três vhosts. Troque `SEU-DOMINIO` antes de colar. O certbot (seção 19) vai adicionar os blocos de TLS e o redirect HTTP→HTTPS automaticamente.

### 17.1 Front ( `SEU-DOMINIO` )

```
# D=SEU-DOMINIO
# cat > /etc/nginx/sites-available/pubmail-front <<EOF2
server {
    listen 80;
    listen [::]:80;
    server_name $D www.$D;
    root /opt/pub-mail/front/vite/dist;
    index index.html;
    client_max_body_size 200m;
    gzip on; gzip_types text/plain text/css application/json application/javascript text/xml application/xml
image/svg+xml; gzip_min_length 1024;
    location = /sw.js { add_header Cache-Control "no-cache"; try_files \$uri =404; }
    location = /manifest.webmanifest { default_type application/manifest+json; add_header Cache-Control "no-
cache"; try_files \$uri =404; }
    location ~* \.(js|css|woff2?|ttf|png|jpg|jpeg|gif|svg|ico|webp)\$ { expires 30d; add_header Cache-Control
"public"; try_files \$uri =404; }
    location / { try_files \$uri \$uri/ /index.html; }
}
EOF2
```

`try_files … /index.html` é o que faz as rotas da SPA (painel, `/webchat/:slug`, `/quiz/:slug`) funcionarem.

As duas linhas `location = …` são do **app instalável (PWA)**: o service worker ( `/sw.js` ) e o manifesto não podem ficar 30 dias em cache (sem elas, o regex do `.js` pegaria o `sw.js` e uma atualização do app demoraria a chegar), e o nginx não conhece o tipo `.webmanifest`. Nada mais do PWA depende do servidor — os arquivos estão em `front/vite/public/` e saem no build.

### 17.2 API ( `api.SEU-DOMINIO` )

```
# cat > /etc/nginx/sites-available/pubmail-api <<EOF2
server {
    listen 80;
    listen [::]:80;
    server_name api.$D;
    client_max_body_size 520m;
    client_body_timeout 600s;
    location ^~ /uploads/ { alias /opt/pub-mail/back/uploads/; expires 365d; add_header Cache-Control "public,
immutable"; try_files \$uri =404; }
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host \$host;
        proxy_set_header X-Real-IP \$remote_addr;
        proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto \$scheme;
        proxy_set_header Upgrade \$http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 300s; proxy_send_timeout 300s;
    }
}
EOF2
```

- O `location ^~ /uploads/` vem **antes** do `location /` : imagens de e-mail e mídia do Telegram saem direto do disco, sem passar pelo Node (é a diferença entre e-mail que abre instantâneo e e-mail lento).
- `client_max_body_size 520m` + `client_body_timeout 600s` : upload de vídeos de produto do webchat (até 500 MB).
- **Nunca** crie um `location` para `/protected/` .

### 17.3 Bloqueio de hosts desconhecidos

```
# cat > /etc/nginx/sites-available/000-default-deny <<'EOF2'
server { listen 80 default_server; listen [::]:80 default_server; server_name _; return 444; }
EOF2
# cat > /etc/nginx/sites-available/000-default-deny-ssl <<'EOF2'
server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;
    server_name _;
    ssl_reject_handshake on;
}
EOF2
```

- O primeiro arquivo recusa, na porta 80, qualquer endereço que não seja um site cadastrado.
- O segundo faz o mesmo na porta 443. Sem ele, o nginx entrega o **primeiro site com HTTPS que encontrar** (normalmente a API) para o IP puro, para subdomínios inventados e para domínio de cliente cujo certificado ainda não saiu. Foi isso que fez um quiz aparecer como "Cannot GET /quiz/…" quando o certificado dele falhou. Com a regra, esses acessos recebem erro de SSL (no Cloudflare, erro 525), que aponta direto para o certificado.
- `ssl_reject_handshake` não precisa de certificado (nginx 1.19.4 ou mais novo), então pode ser criado antes do certbot. Ele não atrapalha a emissão: a validação do Let's Encrypt acontece pela porta 80.

### 17.4 Ativar

```
# rm -f /etc/nginx/sites-enabled/default
# ln -sf /etc/nginx/sites-available/000-default-deny /etc/nginx/sites-enabled/000-default-deny
# ln -sf /etc/nginx/sites-available/000-default-deny-ssl /etc/nginx/sites-enabled/000-default-deny-ssl
# ln -sf /etc/nginx/sites-available/pubmail-front  /etc/nginx/sites-enabled/pubmail-front
# ln -sf /etc/nginx/sites-available/pubmail-api    /etc/nginx/sites-enabled/pubmail-api
# sed -i 's/^\s*# server_tokens off;/\tserver_tokens off;/' /etc/nginx/nginx.conf
# nginx -t && systemctl reload nginx
# curl -s -o /dev/null -w "%{http_code}\n" http://$D/                      # 200
# curl -s -o /dev/null -w "%{http_code}\n" -X POST http://api.$D/auth/login -H 'Content-Type: application/json'
-d '{}'   # 400
```

✅ **Resultado esperado:** `nginx: configuration file /etc/nginx/nginx.conf test is successful` , depois `200` e `400` . Se der `curl: (7) Failed to connect` , o DNS ainda não propagou ou o firewall do provedor está bloqueando a porta 80.

---

## 18. Firewall

```
# ufw --force reset
# ufw default deny incoming
# ufw default allow outgoing
# ufw allow 22/tcp
# ufw allow 80/tcp
# ufw allow 443/tcp
# ufw --force enable
# ufw status
```

MySQL, Redis e os micros escutam em portas internas; mesmo os que escutam em `*` (4000/9999/8000) ficam inacessíveis de fora por causa do firewall.

✅ **Resultado esperado:** `Status: active` com as regras `22/tcp`, `80/tcp` e `443/tcp` como `ALLOW`. Sua sessão SSH continua conectada (a porta 22 foi liberada antes do `enable`).

---

## 19. HTTPS com certbot

Com o DNS propagado e o nginx respondendo na porta 80:

```
# certbot --nginx --non-interactive --agree-tos --email voce@SEU-DOMINIO --redirect \
    -d SEU-DOMINIO -d www.SEU-DOMINIO -d api.SEU-DOMINIO
# nginx -t && systemctl reload nginx
# systemctl enable --now certbot.timer
# certbot renew --dry-run          # "Congratulations, all simulated renewals succeeded"
```

O `--redirect` adiciona o HTTP→HTTPS nos vhosts. O `certbot.timer` renova sozinho (o certificado dura 90 dias).

✅ **Resultado esperado:** "Successfully received certificate" + "Successfully deployed certificate for …" três vezes; `certbot renew --dry-run` termina com "Congratulations, all simulated renewals succeeded". A partir daqui `https://SEU-DOMINIO` abre com cadeado.

### `www` → apex (opcional, recomendado)

Depois do certbot, adicione um bloco 443 para o `www` redirecionar para o domínio principal:

```
# cat >> /etc/nginx/sites-enabled/pubmail-front <<EOF2
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name www.$D;
    ssl_certificate /etc/letsencrypt/live/$D/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/$D/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
    return 301 https://$D\$request_uri;
}
EOF2
# sed -i "s/server_name $D www.$D;\n    root/server_name $D;\n    root/" /etc/nginx/sites-enabled/pubmail-front
# nginx -t && systemctl reload nginx
```

(Se preferir, edite o arquivo à mão: no bloco 443 do front deixe só `server_name SEU-DOMINIO;` e mantenha o novo bloco do `www`.)

---

## 20. Verificação final

Rode da sua máquina (troque o domínio):

```
$ curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" https://SEU-DOMINIO/          # 200
$ curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" http://SEU-DOMINIO/           # 301 → https
$ curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" https://www.SEU-DOMINIO/      # 301 → apex
$ curl -s -o /dev/null -w "%{http_code}\n" https://api.SEU-DOMINIO/uploads/x.png         # 404 (nginx, não o
Node)
$ curl -s https://SEU-DOMINIO/ | grep -o '<title>[^<]*</title>'                          # <title>Pub
Mail</title>
$ curl -s -X POST https://api.SEU-DOMINIO/auth/login -H 'Content-Type: application/json' \
    -d '{"email":"admin@SEU-DOMINIO","password":"UMA-SENHA-FORTE"}' | head -c 120        # {"tokens":{...
```

No servidor:

```
# systemctl is-active pubmail-back pubmail-ai pubmail-email nginx mysql redis-server
# journalctl -u pubmail-back -u pubmail-ai -u pubmail-email --since "-10min" --no-pager | grep -iE
"error|exception" | grep -v "Duplicate DTO"
# mysql -e "SHOW STATUS LIKE 'Threads_connected'"       # ~51
# free -h; df -h /
```

Por fim, entre no painel `https://SEU-DOMINIO` com o SUPER_ADMIN e confira: Dashboard, Configurações → Integrações (sem chaves), Administração → Usuários (só a organização oculta).

> Único aviso esperado nos logs do back: `WARN Duplicate DTO detected: "SaveFlowDto"` (do Swagger; inofensivo).

---

## 21. Pós-instalação

Tudo abaixo é feito **dentro do painel**, por organização (nada é compartilhado entre organizações):

1. **Administração** → **Usuários**: criar as organizações dos clientes e seus usuários.
2. **Configurações** → **Integrações**: chaves de IA (Groq, Mistral, Gemini, OpenRouter…), API key do Resend e o *signing secret* do webhook do Resend. Só o SUPER_ADMIN vê/edita o "IP de borda" e o e-mail do certbot. - No Resend, cadastrar o webhook `https://api.SEU-DOMINIO/webhooks/resend` (eventos delivered/bounced/complained) e verificar o domínio de envio (ex.: `mail.SEU-DOMINIO`).
3. **Telegram** → **Configurações**: cadastrar os bots (token do @BotFather). O webhook do Telegram é configurado automaticamente para `https://api.SEU-DOMINIO/telegram/webhook/<id>`; em migrações de servidor basta "Revalidar" cada bot.
4. **Configurações** → **Pagamentos**: gateway PIX (Mercado Pago ou PushinPay) — token, ativar, "Testar conexão" — e cadastrar no painel do gateway a URL de webhook mostrada na tela (`https://api.SEU-DOMINIO/payments/webhook/<provedor>`). Esse gateway serve o Webchat e o Telegram.
5. **Configurações** → **Domínios**: domínios de webchat/quiz dos clientes (A → `IP-DO-SERVIDOR`, "Verificar DNS" emite o certificado).

---

## 22. WhatsApp (OpenWA em Docker) — opcional

O módulo WhatsApp do Pub Mail usa o **OpenWA** (gateway open source) rodando em Docker, **só em loopback**. O back fala com ele por HTTP, recebe eventos por webhook e empurra tudo para o navegador por WebSocket ( `/realtime` , já coberto pelo `Upgrade` do vhost da API). Instale só se for usar o módulo.

O OpenWA roda **um motor por container**. Use o motor **Baileys**: ele baixa as mídias que os clientes enviam, faz a sincronização inicial do celular e gasta ~100 MB por número. O motor `whatsapp-web.js` **não baixa mídia recebida** desde julho de 2026 (defeito da biblioteca, sem correção) e só fica documentado como alternativa.

### 22.1 Docker

```
apt-get install -y docker.io docker-compose-v2
systemctl enable --now docker
```

### 22.2 OpenWA com o motor Baileys (recomendado)

```
mkdir -p /opt/openwa-baileys/data && cd /opt/openwa-baileys
umask 077
cat > .env <<EOF2
API_MASTER_KEY=$(openssl rand -hex 32)
API_KEY_PEPPER=$(openssl rand -hex 32)
ENGINE_TYPE=baileys
PORT=2785
NODE_ENV=production
AUTO_START_SESSIONS=true
MAX_CONCURRENT_SESSIONS=40
SESSION_DATA_PATH=/app/data/sessions
DATABASE_TYPE=sqlite
WEBHOOK_SSRF_PROTECT=true
SSRF_ALLOWED_HOSTS=host.docker.internal
RESOLVE_LID_TO_PHONE=true
SEND_PACING_ENABLED=false
BAILEYS_MARK_ONLINE_ON_CONNECT=false
BAILEYS_BROWSER_NAME=Ubuntu
SIMULATE_TYPING=false
WEBHOOK_MEDIA_INLINE_MAX_BYTES=8388608
WEBHOOK_MAX_PAYLOAD_BYTES=12582912
RATE_LIMIT_SHORT_LIMIT=100
RATE_LIMIT_MEDIUM_LIMIT=3000
RATE_LIMIT_LONG_LIMIT=100000
# mídia vai em base64 (+33 %); o padrão de 25mb recusa vídeo acima de ~18 MB (resposta 404 "Cannot POST")
BODY_SIZE_LIMIT=100mb
EOF2
umask 022
cat > docker-compose.yml <<'EOF2'
services:
  openwa-baileys:
    image: ghcr.io/rmyndharis/openwa:0.23.7
    container_name: openwa-baileys
    restart: unless-stopped
    env_file: .env
    ports: ["127.0.0.1:2786:2785"]
    volumes: ["./data:/app/data"]
    extra_hosts: ["host.docker.internal:host-gateway"]
    mem_limit: 2g   # vídeo grande ocupa ~3 cópias em memória no envio
    pids_limit: 512
EOF2
docker compose pull && docker compose up -d
sleep 20 && curl -s http://127.0.0.1:2786/api/health     # {"status":"ok"}
```

- `BAILEYS_MARK_ONLINE_ON_CONNECT=false` é importante: sem isso o celular do cliente para de receber notificações enquanto o gateway estiver conectado.
- `SIMULATE_TYPING=false`: o Pub Mail faz o próprio "digitando…" no bot; o do OpenWA atrasaria cada mensagem enviada pelo painel.
- Os dois `WEBHOOK_*` fazem mídia de até 8 MB chegar direto no webhook; acima disso o Pub Mail baixa pela rota de mídia do gateway.

- Os `RATE_LIMIT_*` sobem o limite da API do gateway (o padrão é 100 requisições por minuto). O único cliente é o back do Pub Mail, que atende todas as organizações e carrega dezenas de fotos de uma vez; com o padrão, avatares e mídias de status falham com 429. Aplique o mesmo no gateway whatsapp-web.js, se usar.
- **Atualize o OpenWA só manualmente** (troque a tag da imagem, `docker compose pull && up -d`): o projeto lança várias versões por semana com mudanças incompatíveis.

> **Alternativa (não recomendada): motor whatsapp-web.js.** Mesmo passo a passo em `/opt/openwa`, com `ENGINE_TYPE=whatsapp-web.js`, porta `127.0.0.1:2785:2785`, `mem_limit: 3g`, `pids_limit: 2048`, `shm_size: 1g` (Chrome headless, 300–500 MB por número) e as variáveis `OPENWA_URL` / `OPENWA_API_KEY` no back. O Pub Mail aceita os dois gateways ao mesmo tempo (motor por conexão, com botão de migrar em Configurações), mas neste motor as mídias recebidas não chegam.

### 22.3 Firewall (o container precisa alcançar o back)

O webhook do OpenWA chega no back pelo endereço `host.docker.internal:8000`. O `ufw` bloqueia esse tráfego por padrão:

```
ufw allow from 172.16.0.0/12 to any port 8000 proto tcp comment 'openwa -> pubmail back'
docker exec openwa-baileys node -e
"fetch('http://host.docker.internal:8000/auth/me').then(r=>console.log(r.status))"   # 401 = alcançou
```

### 22.4 `.env` do back

```
KEY=$(grep ^API_MASTER_KEY= /opt/openwa-baileys/.env | cut -d= -f2)
cat >> /opt/pub-mail/back/.env <<EOF2

# ---- WhatsApp (OpenWA em Docker, só loopback) ----
OPENWA_BAILEYS_URL=http://127.0.0.1:2786/api
OPENWA_BAILEYS_API_KEY=$KEY
WHATSAPP_DEFAULT_ENGINE=baileys
OPENWA_WEBHOOK_BASE=http://host.docker.internal:8000
# OPENWA_URL / OPENWA_API_KEY   # só se também rodar o gateway whatsapp-web.js (porta 2785)
# WHATSAPP_MEDIA_QUOTA_MB=25600  # cota de mídia por organização (padrão 2 GB é pouco: 2 números ≈ 250 MB/dia ×
90 dias; produção usa 25 GB)
EOF2
mkdir -p /opt/pub-mail/back/protected/whatsapp && chmod 750 /opt/pub-mail/back/protected/whatsapp
mkdir -p /opt/pub-mail/back/protected/imports && chmod 750 /opt/pub-mail/back/protected/imports   # planilhas
de importação de clientes (apagadas ao terminar)
systemctl restart pubmail-back
```

#### Criptografia das mensagens e mídias (recomendado)

O back guarda o texto das mensagens, as prévias, as legendas de status e os arquivos de mídia **cifrados** (AES-256-GCM, uma chave por organização). Quem abrir o banco, um dump ou a pasta `protected/whatsapp` vê só dados embaralhados; no painel tudo aparece normal. A chave mestra é gerada no próprio servidor e nunca é impressa:

```
K=$(openssl rand -base64 32)
printf '\nWHATSAPP_DATA_KEYS=1:%s\n' "$K" >> /opt/pub-mail/back/.env
printf 'WHATSAPP_DATA_KEYS=1:%s\n' "$K" >> /root/pubmail.secrets
unset K; chmod 600 /opt/pub-mail/back/.env /root/pubmail.secrets
systemctl restart pubmail-back
journalctl -u pubmail-back -n 50 --no-pager | grep Criptografia   # "LIGADA"
```

> ⚠️ **Sem essa chave as mensagens não abrem mais.** Guarde uma cópia FORA do servidor (gerenciador de senhas): `grep WHATSAPP_DATA_KEYS /root/pubmail.secrets`. Nunca troque o valor da chave `1`; para trocar de chave, acrescente uma nova com número maior (`1:<antiga>,2:<nova>`) — as organizações novas passam a usar a `2` e a `1` continua abrindo o que já existe.

Mensagens e arquivos gravados antes da chave existir são cifrados sozinhos em segundo plano (90 s depois do boot e a cada 30 min; o log mostra `Cifrados: …`). Sem a variável, o sistema funciona normalmente, só sem cifrar.

**Notificações push (Web Push).** O painel manda notificações para o celular/computador dos atendentes (mensagem nova, fila, pagamento…). Precisa de um par de chaves VAPID no `back/.env` — sem elas o push fica desligado e só os avisos dentro do painel funcionam:

```
cd /opt/pub-mail/back
node -e "const w=require('web-push');const
k=w.generateVAPIDKeys();process.stdout.write('WEB_PUSH_VAPID_PUBLIC_KEY='+k.publicKey+'\nWEB_PUSH_VAPID_PRIVATE
_KEY='+k.privateKey+'\nWEB_PUSH_SUBJECT=mailto:SEU-EMAIL\n')" > /tmp/vapid.env
cat /tmp/vapid.env >> .env; cat /tmp/vapid.env >> /root/pubmail.secrets; rm -f /tmp/vapid.env
systemctl restart pubmail-back && journalctl -u pubmail-back -n 100 --no-pager | grep "Web Push"   # → "Web
Push: LIGADO"
```

Troque `SEU-EMAIL` pelo e-mail do responsável (os serviços de push usam esse contato). **Não troque as chaves depois**: cada aparelho inscrito guarda a chave pública; uma chave nova obriga todo mundo a ativar de novo.

## 22.5 Backup do volume (sessões WhatsApp)

O volume `data/` de cada gateway guarda as credenciais das sessões pareadas. Sem backup, um disco perdido obriga todo cliente a escanear o QR de novo. Script + timer diário (04:30 UTC, 7 dias de retenção) cobrindo os gateways que existirem.

**Limpeza do gateway (Baileys):** o gateway guarda uma cópia **em claro** de todas as mensagens (tabela `messages` do `data/openwa.sqlite`, com a mídia em base64) e dos webhooks entregues. O Pub Mail já guarda tudo cifrado (§22.4), então na mesma pausa do backup, antes do `tar`, o script `cleanup.py` apaga o que tem mais de 7 dias (`secure_delete` zera as páginas, o índice de busca é refeito, VACUUM só quando ≥ 20 % do arquivo está livre, cerca de 30 s). Não toca em `baileys_stored_messages` (responder/reagir/editar/apagar; o gateway já limita às últimas 5.000), nos status (o gateway apaga os vencidos) nem nas sessões. Copie o script do repositório (`docs/tools/openwa_cleanup.py`):

```
mkdir -p /opt/openwa-baileys/bin
scp docs/tools/openwa_cleanup.py <servidor>:/opt/openwa-baileys/bin/cleanup.py   # na sua máquina
chmod 755 /opt/openwa-baileys/bin/cleanup.py   # roda como o dono do banco (uid do container), precisa ler o
script
# teste numa cópia (sem pausar nada):
#   python3 /opt/openwa-baileys/bin/cleanup.py --db /root/copia/openwa.sqlite --no-pause --days 7
```

Backup + limpeza:

```
mkdir -p /opt/backups/openwa /opt/openwa/bin && chmod 700 /opt/backups
cat > /opt/openwa/bin/backup.sh <<'EOF2'
#!/bin/bash
set -euo pipefail
DEST=/opt/backups/openwa; mkdir -p "$DEST"; STAMP=$(date +%F)
backup_one() {  # $1=container $2=pasta do compose $3=prefixo [$4=script de limpeza]
  [ -d "$2" ] || return 0
  docker pause "$1" >/dev/null 2>&1 || true
  if [ -n "${4:-}" ] && [ -x "$4" ]; then python3 "$4" --no-pause --days 7 >/dev/null 2>&1 || true; fi
  if tar -czf "$DEST/$3-$STAMP.tar.gz" -C "$2" data .env docker-compose.yml; then chmod 600
"$DEST/$3-$STAMP.tar.gz"; fi
  docker unpause "$1" >/dev/null 2>&1 || true
}
backup_one openwa /opt/openwa openwa-data
backup_one openwa-baileys /opt/openwa-baileys openwa-baileys-data /opt/openwa-baileys/bin/cleanup.py
find "$DEST" -name 'openwa-*-*.tar.gz' -mtime +7 -delete
EOF2
chmod 700 /opt/openwa/bin/backup.sh
printf '[Unit]\nDescription=Backup OpenWA\n[Service]\nType=oneshot\nExecStart=/opt/openwa/bin/backup.sh\n' >
/etc/systemd/system/openwa-backup.service
printf '[Unit]\nDescription=Backup diário OpenWA\n[Timer]\nOnCalendar=*-*-*
04:30:00\nPersistent=true\n[Install]\nWantedBy=timers.target\n' > /etc/systemd/system/openwa-backup.timer
systemctl daemon-reload && systemctl enable --now openwa-backup.timer
```

Resultado da limpeza em `/var/log/openwa-cleanup.log` ( `limpeza ok: N mensagens, … arquivo X MB -> Y MB; check=ok; T s` ). A pausa noturna fica em ~2–4 min (limpeza lendo o banco do disco + `tar` ); durante ela o WhatsApp segura as mensagens e entrega quando o container volta. O healthcheck do Docker pode marcar `unhealthy` por uma rodada logo depois — volta sozinho. Os backups antigos (ainda com mensagens em claro) saem pela rotação de 7 dias.

### 22.6 Operação

```
cd /opt/openwa-baileys && docker compose ps    # healthy?
docker compose logs --tail 100                 # logs do gateway
docker stats --no-stream                       # RAM dos gateways
journalctl -u pubmail-back -n 100 | grep -i whatsapp
curl -s "https://api.<SEU_DOMINIO>/realtime/?EIO=4&transport=polling"   # WebSocket do painel: responde
0{"sid":…}
```

Depois disso, no painel: **Administração** → **Usuários** libera o módulo **WhatsApp** por usuário; o cliente conecta em **WhatsApp** → **Configurações** (tutorial em `WhatsApp - Como conectar.pdf` ). Em **Configurações** → **Diagnóstico da conexão** o painel mostra se o gateway responde, o motor em uso e as mídias recebidas nas últimas 24 h.

## 23. Atualizando o sistema

Regras de ouro: **nunca** envie `.env` ; confira antes se o servidor não tem versão mais nova do arquivo ( `md5sum` local × servidor); reinicie o back **só** se o build passou sem `error TS` .

### 23.1 Front (mudou algo em `front/vite/src` )

Build numa pasta temporária + troca atômica: se o build falhar, o site continua no ar com a versão anterior.

```
$ tar -czf - front/vite/src | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail'
$ ssh pubmail-prod 'cd /opt/pub-mail/front/vite && export NODE_OPTIONS=--max-old-space-size=4096 \
    && rm -rf dist_new && yarn build --outDir dist_new \
    && [ -f dist_new/index.html ] && { rm -rf dist_prev; mv dist dist_prev; mv dist_new dist; } && echo
SWAP_OK'
```

Os usuários precisam de *hard refresh* (Cmd/Ctrl+Shift+R) — o nome dos bundles muda. Se mudou dependência (`package.json`), rode `unset CI; yarn install` antes do build.

### 23.2 Back (mudou `back/src`, `schema.prisma` ou migration nova)

```
$ tar -czf - back/src back/prisma | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail'
$ ssh pubmail-prod 'bash -s' <<'EOF2'
set -eo pipefail; cd /opt/pub-mail/back
yarn prisma migrate deploy && yarn prisma generate        # só se houve migration/schema
yarn nest build > /tmp/nest-build.log 2>&1 || true
ERR=$(grep -cE "error TS|Found [0-9]+ error" /tmp/nest-build.log || true); echo "TS_ERRORS=$ERR"
[ "$ERR" = "0" ] || { tail -30 /tmp/nest-build.log; exit 1; }
test -f dist/src/main.js && systemctl restart pubmail-back && sleep 8 && systemctl is-active pubmail-back
journalctl -u pubmail-back -n 30 --no-pager | grep -iE "error|successfully started" | tail -5
EOF2
```

Migration nova = pasta `back/prisma/migrations/<timestamp>_nome/migration.sql` (timestamp maior que o último) + alteração correspondente no `schema.prisma`. Toda tabela nova deve ser criada com `DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci`.

### 23.3 Micros

```
$ tar -czf - ai-micro-services/src | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail && systemctl restart
pubmail-ai'
$ tar -czf - email-micro-services/*.js | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail && systemctl restart
pubmail-email'
```

(Se mudou dependência: `unset CI; yarn install` na pasta antes do restart.)

### 23.4 Plugins WordPress (mudou `wordpress/tradutor-invisivel` ou `wordpress/push-in-play`)

Tradutor Invisível: compile os scripts na sua máquina (kit em `wordpress/tradutor-invisivel-dev`, com `npm install` feito uma vez) e envie a pasta do plugin. Push In Play: só enviar a pasta (`tar -czf - wordpress/push-in-play | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail'`) depois de subir `Version:` / `PIP_VERSION`. Não precisa reiniciar nada: o back lê a pasta a cada download e a versão pelo cabeçalho do arquivo principal.

```
$ (cd wordpress/tradutor-invisivel-dev && npm test)          # compila assets/*.min.js e roda os testes
$ tar -czf - wordpress/tradutor-invisivel | ssh pubmail-prod 'tar -xzf - -C /opt/pub-mail'
```

Suba o `Version:` do plugin (e o `TRINV_VERSION`) a cada entrega: os sites dos clientes veem a versão nova em até 6 h e o WordPress atualiza sozinho.

---

# 24. Operação do dia a dia

### Logs

```
# systemctl status pubmail-back pubmail-ai pubmail-email --no-pager | grep Active
# journalctl -u pubmail-back -f                    # ao vivo
# journalctl -u pubmail-back --since "-1h" --no-pager | grep -i error
# tail -f /var/log/nginx/error.log
```

### SQL sem expor a senha

```
# mysql -upubmail -p"$(grep ^DB_PASS= /root/pubmail.secrets | cut -d= -f2)" -h127.0.0.1 pubmail -e "SELECT
COUNT(*) FROM users"
```

Scripts grandes: copie o `.sql` para `/tmp` e use `mysql ... < /tmp/arquivo.sql`.

### Backup

```
# mkdir -p /root/backups
# mysqldump -upubmail -p"$(grep ^DB_PASS= /root/pubmail.secrets | cut -d= -f2)" -h127.0.0.1 --single-
transaction pubmail \
    | gzip > /root/backups/pubmail-$(date +%F).sql.gz
# tar -czf /root/backups/files-$(date +%F).tgz /opt/pub-mail/back/uploads /opt/pub-mail/back/protected
/root/pubmail.secrets /opt/pub-mail/back/.env /opt/pub-mail/*/.env /opt/pub-mail/front/vite/.env
```

Agende no cron ( `crontab -e` ) e copie os arquivos para fora do servidor. O que precisa de backup: **banco**, **uploads/**, **protected/**, **segredos e `.env`**. O código está no repositório.

### Trocar a senha do SUPER_ADMIN

Basta rodar o seed de novo (seção 13) com a nova `SEED_ADMIN_PASSWORD`.

### Reiniciar tudo

```
# systemctl restart pubmail-back pubmail-ai pubmail-email && systemctl reload nginx
```

---

# 25. Solução de problemas

| Sintoma | Causa | Solução |
|---|---|---|
| `yarn install` do back falha com "engine node is incompatible… Expected >=22" ou `yarn --version` mostra `1.22` | O corepack resolveu o Yarn clássico (pasta sem `packageManager` ) ou o Node é 20 | Node 22 (seção 5); confirme `"packageManager": "yarn@4.10.3"` no `package.json` da pasta. |
| `yarn install` → "Failed with errors" em 0 s | Variável `CI` definida → Yarn em modo imutável | `unset CI` |
| Back não sobe: "STRIPE_SECRET_KEY não configurada" | Variável vazia no `.env` | `STRIPE_SECRET_KEY=sk_test_disabled_placeholder` |

| Sintoma | Causa | Solução |
|---|---|---|
| "Too many connections" no MySQL | `DATABASE_POOL_LIMIT` ausente | `DATABASE_POOL_LIMIT=10` no `back/.env` e reiniciar o back |
| Buffer pool continua 128 MB | `.cnf` com permissão 600 (umask 077) → MySQL ignora | `chmod 644 /etc/mysql/mysql.conf.d/zz-pubmail.cnf && systemctl restart mysql` |
| nginx `403` / `Permission denied` em `/uploads/...` | pastas criadas com 700 | `chmod 755 /opt/pub-mail/back/uploads /opt/pub-mail/back/uploads/*` |
| Migration falha com erro 3780 (FK) | tabela criada sem collation explícita (default `utf8mb4_0900_ai_ci`) | banco e tabelas em `utf8mb4_unicode_ci`; `yarn prisma migrate resolve --rolled-back <nome>` e reaplicar |
| "Verificar DNS" de domínio de cliente dá erro `WEBCHAT_DOMAIN_BIND_PROVIDER inválido` | valor diferente de `CERTBOT/NONE/DISABLED/MOCK` | `WEBCHAT_DOMAIN_BIND_PROVIDER=CERTBOT` |
| `/ads.txt` do domínio de cliente quebrado | `WEBCHAT_BACKEND_INTERNAL_URL` ausente (default `:3000`) | definir `http://127.0.0.1:8000` |
| Disparo de e-mail fica "enviando" para sempre | `NEST_WEBHOOK_URL` sem o sufixo `/webhooks/email` | corrigir no `email-micro-services/.env` e reiniciar `pubmail-email` |
| Aberturas/cliques de e-mails novos não contam (pixel 204, clique 403) | `TRACKING_SIGN_SECRET` do micro ausente ou diferente do `JWT_ACCESS_SECRET` do back | igualar os dois valores e reiniciar `pubmail-email` |
| `sshd -T` mostra `passwordauthentication yes` mesmo com o drop-in | drop-in com nome `99-` perde para `50-cloud-init.conf` | renomear para `00-pubmail.conf` |
| Deploy "passou" mas o código antigo continua rodando | `nest build` emite `dist/` mesmo com erro TS, e `tail` / `grep` num pipe escondem falhas | sempre `grep -c "error TS"` = 0 antes do restart; `set -o pipefail` nos scripts |
| Webchat com IA responde "Falha ao processar mensagem" e as chaves Groq aparecem "Inválida" | provedor descontinuou o modelo (404 `model_not_found`) | atualizar `defaultModel` / `fallbackModel` em `ai-micro-services/src/providers/<provedor>-adapter.js`; limpar o status no Redis: `redis-cli --scan --pattern 'pubmail:ai_keys:*'` → `del` das chaves com `status=invalid` |
| Build do front morre sem mensagem | falta de memória | swap (seção 6) + `NODE_OPTIONS=--max-old-space-size=4096` |
| certbot falha "Timeout during connect" | DNS ainda não propagado ou nuvem laranja da Cloudflare | esperar `dig` responder o IP; deixar os registros em DNS only |

---

## 26. Instalando uma segunda cópia

Para outra empresa/domínio, **repita o tutorial inteiro em outro servidor**: banco, segredos, `.env`, seed, contas de Resend/Telegram/gateway — nada é compartilhado. Só o código é o mesmo. Checklist do que muda entre instalações:

- `SEU-DOMINIO` e `IP-DO-SERVIDOR` (DNS, `.env` do back/micros/front, vhosts, certbot, seed).
- Todos os valores de `/root/pubmail.secrets` (gerados de novo).
- `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` / `SEED_CERTBOT_EMAIL`.
- As integrações cadastradas no painel (seção 21).

Guarde os acessos de cada servidor num arquivo **fora do repositório** (ex.: `servers.json`, já no `.gitignore`).

---

## Glossário rápido

- **VPS** — servidor virtual alugado (uma máquina Linux só sua na nuvem).
- **root** — o usuário administrador do Linux; pode tudo.
- **SSH** — protocolo para abrir um terminal remoto no servidor. **Chave SSH** = par de arquivos (privado na sua máquina, público no servidor) que substitui a senha.
- **DNS / registro A / CNAME** — o "catálogo" que traduz `SEU-DOMINIO` para o IP do servidor. `A` aponta para um IP; `CNAME` aponta para outro nome.
- **Nuvem cinza / laranja (Cloudflare)** — cinza = só DNS (o tráfego vai direto ao seu servidor); laranja = passa pelo proxy da Cloudflare.
- **nginx** — o servidor web: recebe as conexões HTTPS, entrega os arquivos do front e repassa a API para o Node.
- **vhost (virtual host)** — um bloco `server { … }` do nginx, um por domínio.
- **certbot / Let's Encrypt** — emite e renova os certificados HTTPS de graça.
- **systemd / unit** — o gerenciador de serviços do Linux; uma *unit* é a "ficha" de um serviço (`pubmail-back.service`), que o mantém rodando e reinicia se cair.
- `.env` — arquivo com as variáveis de configuração de um serviço (senhas, URLs). Um por serviço, nunca versionado.
- **Migration** — script SQL versionado que cria/altera tabelas. `prisma migrate deploy` aplica as que faltam, na ordem.
- **Seed** — script que insere os dados iniciais (usuário administrador, configurações da plataforma).
- **Build** — compilar o código (TypeScript → JavaScript no back; React → arquivos estáticos no front).
- **Yarn / corepack** — gerenciador de pacotes do Node e a ferramenta que garante a versão certa dele.
- **SPA** — *single-page application*: o front é um único `index.html` + JS; por isso o nginx precisa do `try_files … /index.html`.
- **Webhook** — URL pública que um serviço externo (Telegram, Resend, Mercado Pago) chama para avisar de um evento.

---

*Última revisão: 26/09/2026 — baseado na instalação de produção em `reveleonline.com` (Hostinger, Ubuntu 24.04.5, Node 22.23, Yarn 4.10.3, nginx 1.24, MySQL 8.0.46, Redis 7.0.15, certbot 2.9, ffmpeg 6.1).*